

1. Introdução

- O objetivo da prova foi implementar técnicas clássicas de PDI “do zero”.
- O foco principal foi:
 - equalização e especificação com histograma;
 - filtragem linear;
 - filtragem espacial;

Questão 1: Histograma

2. Conceito de histograma

- O histograma mostra a distribuição dos níveis de cinza da imagem
 - para uma imagem RGB, normalmente o histograma é calculado separadamente para cada canal
- Histograma é ótimo para saber se tem alto ou baixo contraste
- **Contraste:** é a diferença entre os níveis de intensidade presentes em regiões distintas da imagem.
- Imagens com baixo contraste possuem pixels concentrados em uma faixa pequena de intensidade.

3. Equalização de histograma

- Objetivo: espalhar os níveis de cinza por toda a faixa dinâmica da imagem
- A equalização usa:
 - histograma normalizado
 - PDF (Probability Density Function - $pr(kr)^1$): representa a probabilidade de cada nível de cinza aparecer na imagem
 - no nosso caso = quantidade de pixels no nível de cinza X / total de pixels da imagem
 - CDF (Cumulative Distribution Function): função acumulada que acumula as probabilidades da PDF até um determinado nível de cinza, ou seja: pega as probabilidades da PDF; soma tudo do nível 0 até o nível k
- **Comentários:**
 - A transformação serve para transformar os níveis de cinza da imagem original em novos níveis mais espalhados, aumentando o contraste.
 - Comentar sobre o np.zeros -> array nulo

¹ p → probabilidade; r → variável de intensidade da imagem original; rk → um nível específico de cinza;

4. Equalização - implementação

- Implementei manualmente:
 - cálculo do histograma;
 - normalização ao mesmo tempo que obtém PDF;
 - cálculo da CDF : é basicamente um somatório de todas as PDFs
 - $\text{cdf}[0] = \text{pr}[0]$: A CDF começa no primeiro valor da PDF, pois no nível inicial ainda não há probabilidades anteriores para acumular
 - transformação
 - $s_k \rightarrow$ novo nível de cinza depois da transformação.
 - $L \rightarrow$ quantidade total de níveis de cinza.
 - Quanto mais rápido a CDF cresce em uma região:
 - maior é a concentração de pixels naquela faixa;
 - e mais a transformação vai espalhar esses níveis.
- Depois cada pixel foi mapeado para um novo valor de intensidade.

5. Resultados da equalização

- **Comentários:**
 - `.convert('L')` converte a imagem para escala de cinza
 - `plt.tight_layout()`: ajusta automaticamente os espaços entre os gráficos da figura.
 - O caractere `\` é usado para sequências especiais em Python por isso dos warnings
- Os gráficos da CDF que são normalizado e o pré-transformação que só mostra a mesma coisa em dados brutos
- As imagens “pollen” e “messi” ganharam contraste.
- O histograma ficou mais espalhado.
- Detalhes antes escondidos ficaram mais visíveis.
- Ex de cálculo de redistribuição: no histograma do pollen tem uma quantidade boa (cerca de 8% de pixels no nível 90), na CDF teríamos 0,08, na transformação $255 \cdot 0.08 = 20.4$

6. Por que o histograma não fica perfeitamente plano?

- Porque a imagem é discreta.
- Existe arredondamento dos níveis de cinza.
- Vários níveis podem cair no mesmo valor de saída.
- Isso cria:

- picos;
- bins vazios;
- pequenas irregularidades.
- Em resumo: o gráfico é redistribuído horizontalmente apenas variando as colunas, para planificar, precisaria de uma CDF com integral e não somatória que fizesse uma variação entre as colunas, não sendo apenas uma somatória

7. Especificação de histograma — teoria

- Diferente da equalização, aqui usamos uma imagem de referência.
- O objetivo é fazer a imagem de entrada “copiar” a distribuição tonal da referência.
- Foi usada:
 - CDF da entrada;
 - CDF da referência;
 - composição de transformações

8. Especificação — implementação

- **Comentários:**
 - Pega uma intensidade X da imagem original e calcula seu valor na CDF. Depois procura na imagem de referência qual intensidade Y possui um valor de CDF equivalente ao de X . Quando encontra, a intensidade X é mapeada para Y .
 - Ex.: $T(x)$ $T(100)=0.7$ $G(y) = 0.7$ $y=180$ $100 \rightarrow 180$ (então: todo pixel com intensidade 100 será substituído por 180)
 - Primeiro obtém o G_{inv} , que é a transformada inversa, ou seja, para cada valor transformado s : procura qual intensidade z da referência possui valor mais próximo.
 - Depois, na LUT: aqui acontece a especificação propriamente dita: Primeiro:
 - $T(r)$ transforma a intensidade da imagem original. Depois:
 - G^{-1} encontra a intensidade correspondente da referência..
- O mapeamento foi feito usando vizinho mais próximo.

9. Conclusão da Questão 1

- Equalização melhora contraste global.
- Especificação permite controlar melhor o “estilo” da imagem.
- A equalização pode ser vista como um caso particular da especificação, usando um histograma uniforme como alvo.

Questão 2 — Filtros Lineares Homogêneos

10. Objetivo

- Implementar filtros espaciais para suavização.
- Comparar:
 - filtro de média;
 - filtro Gaussiano.
- Estudar:
 - tamanho do kernel;
 - efeito nas bordas;
 - padding.

PARTE A

11. Filtro de média — teoria

- **Filtros** são operações que calculam novos valores de pixels a partir de um conjunto de pixels da imagem original
- **Linear:** a saída é obtida por uma combinação linear dos pixels da vizinhança;
- **Homogêneo:** o mesmo kernel é aplicado igualmente em toda a imagem.
- O **kernel** é uma matriz de coeficientes que define como cada posição da vizinhança influenciará o novo valor calculado.
- Resultado: reduz ruído; mas borra bastante as bordas.
- **Comentários:** k é o tamanho do kernel, e usa $1/k^2$ só por ser um quadrado de $k \times k$ de pixels

12. Filtro Gaussiano — teoria

- Os pesos seguem distribuição Gaussiana:
- Pixels próximos ao centro têm mais influência.
- Isso preserva melhor as bordas.
- **Comentários:** h é valor do kernel; x e y a distância da posição até o centro do kernel; σ é o desvio padrão; pares dão o tamanho do kernel e o valor de sigma
- O **sigma** controla o espalhamento da Gaussiana, ou seja, o quanto o filtro desfoca a imagem.

13. Implementação dos kernels

- Os kernels foram criados manualmente.

- Depois normalizados para:
 - soma igual a 1;
 - simetria da máscara.
- Foram testados tamanhos:
 - 3×3 ;
 - 7×7 ;
 - 15×15 .
- **Comentários:**
 - O `_` no primeiro for é só para aproveitar os pares das gaussianas
 - Os **assert** servem para verificar automaticamente se uma condição está correta durante a execução do código

assert abs(km.sum() - 1.0) < 1e-10

km → kernel da média.

km.sum() → soma dos pesos do kernel.

Verifica se: a soma dos pesos é aproximadamente 1.

Se for verdadeiro: o kernel está corretamente normalizado.

assert np.allclose(km, km.T)

km.T → transposta do kernel.

np.allclose → compara duas matrizes.

Verifica se: o kernel é igual à sua transposta.

Se for verdadeiro: o kernel da média é simétrico.

assert abs(kg.sum() - 1.0) < 1e-10

kg → kernel Gaussiano.

Verifica se: a soma dos pesos é aproximadamente 1.

Se for verdadeiro: o kernel Gaussiano está corretamente normalizado.

assert np.allclose(kg, kg.T)

Verifica se:

o kernel é igual à sua transposta.

Se for verdadeiro:

isso confirma que o kernel é simétrico

O que a suavização me diz:

Filtro de média: se ele não fosse simétrico, você estaria suavizando mais em uma direção do que na outra (o que quebraria a ideia de “média uniforme”). A simetria garante que a janela é realmente homogênea.

Filtro gaussiano: a simetria é ainda mais fundamental, porque a Gaussiana depende apenas da distância ao centro ($x^2 + y^2$), não da direção. Se não fosse simétrico, o “desfoque” deixaria de ser circular e viraria algo deformado (tipo elipse ou viés direcional).

- Na exibição dos kernels: não existe imagem original, as cores são geradas artificialmente a partir dos valores do kernel

14. Janela deslizante

- **Comentários:**

- **Padding** é o processo de **adicionar pixels artificiais ao redor da imagem** antes de aplicar um filtro, para permitir que a janela do kernel “caiba” também nas bordas.
- Implementa um filtro linear espacial por janela deslizante (padding)
- **Caso 1: zeros:** `np.pad(..., mode='constant', constant_values=0)`
 - Borda vira preto (0)., ao redor no caso
- **Caso 2: reflect:** `np.pad(..., mode='reflect')`
 - A borda é “espelhada”, evitando artefato.

5	4	5	vem de	
2	1	2		1 2
5	4	5		4 5
- Por que isso existe? Porque o kernel precisa de vizinhos até nas bordas, e a imagem não tem pixels fora dela.
- Janela deslizante percorre cada pixel da imagem
- Sem padding:
 - você perde informação nas bordas
 - a imagem de saída fica menor (efeito de “encolhimento”)
 - bordas ficam mal definidas ou simplesmente ignoradas

- O `np.pad` foi usado para tratar bordas.

15. Padding: zeros vs reflexão

- Padding com zeros:
 - cria escurecimento artificial nas bordas.
- Padding por reflexão:
 - usa pixels espelhados;
 - gera resultados mais naturais.
- Na prática, reflexão costuma ser melhor.

PARTE B

16. Comparação dos filtros

- O filtro de média produz borramento uniforme.
- O Gaussiano suaviza sem destruir tanto as bordas.
- Quanto maior o kernel e o sigma: maior o desfoque.

Questão 3 — Efeito Bokeh / Tilt-Shift

17. Objetivo

- Simular profundidade de campo.
- Manter o assunto principal nítido.
- Desfocar o fundo da imagem.
- Conceitos:
 - Efeito Bokeh: É o **desfoque estético do fundo fora de foco**, causado principalmente pela abertura da lente.
 - Quando um ponto da cena não está exatamente no plano de foco da câmera, ele não vira um ponto perfeito no sensor. Em vez disso, vira um círculo de desfoque chamado circle of confusion.
 - Isso acontece porque: a lente só consegue focar perfeitamente em uma distância por vez; pontos mais próximos ou mais distantes são projetados “espalhados” no sensor
 - Efeito tilt shift: É uma técnica (ótica ou digital) que cria uma **faixa estreita de foco e desfoca o resto da imagem propositalmente**.

18. Ideia matemática

- Criar uma máscara:

- **Círculo** (modo retrato): 1 dentro de um raio ao redor do centro, 0 fora.
- **Faixa horizontal** (efeito tilt-shift): 1 em uma faixa horizontal central, 0 fora.
- Depois suavizar essa máscara com Gaussiano.

19. Pipeline do efeito + Implementação

- Etapas:
 - criar máscara;
 - suavizar máscara;
 - desfocar imagem inteira;
 - combinar imagem original e desfocada.
- **Comentários:**
 - O “efeito de adesivo colado” (às vezes chamado de cut-out look) é quando um objeto parece recortado e colocado por cima da imagem, sem integração natural com o ambiente.

Para mascara do círculo:

- Definição padrão: centro = meio da imagem; raio = 1/4 da menor dimensão
- **dist = $\sqrt{(i - cy)^2 + (j - cx)^2}$** Isso calcula a distância do pixel até o centro.
- Retorna um R array binário de dentro (1) e fora (0)

Para a mascara de faixa

- Mesma ideia só que a conta é mais simples

Gaussiana

- Apenas aplica a gaussiana que criamos lá em cima para suavizar as bordas

○ **Mesclagem:**

- I: imagem original (nítida)
- I~: imagem desfocada (aplicação de blur Gaussiano)
- R: máscara binária (região de foco “dura”)
- R~: máscara suavizada (transição gradual entre foco e desfoque)
- 1-R~: complemento da máscara (regiões desfocadas)

20. Importância da suavização da máscara

- Sem suavização:
 - aparece um corte artificial.

- Com suavização:
 - a transição fica natural;
 - o efeito lembra profundidade de campo real.

21. Influência do sigma

- Sigma pequeno:
 - desfoque leve;
 - efeito mais sutil.
- Sigma grande:
 - fundo muito borrado;
 - destaque maior para o objeto principal.

Conclusão Final

- Em ambos os casos, a etapa de suavização da máscara ($R \rightarrow \sim R$) é fundamental: sem ela, a borda entre a região nítida e a desfocada seria um corte abrupto, dando a impressão de uma colagem grosseira em vez de um degradê natural de profundidade de campo.